

Instituto Superior Incorporado N° 4021

ICOP Santa Fe

**Dependiente del Servicio Provincial de Enseñanza Privada
Ministerio de Educación de la Provincia de Santa Fe**

PROGRAMACIÓN I – 2DO AÑO

Profesor: Javier Fernando Imvinkelried

TRABAJO PRÁCTICO INTEGRADOR

Lógica y Fundamentos de Programación con PHP

 **SEOM LAB**

Sistema de Estacionamiento Medido



Sistema de Estacionamiento Medido

Programación I — 2.º año | Desarrollo de Software

Prof. Javier Imvinkelried

EL DESAFÍO

En esta actividad vamos a comenzar a trabajar con un problema más cercano a los que podemos encontrar en un sistema informático real.

Vamos a desarrollar, utilizando **PHP puro**, una versión educativa y simplificada de la lógica de un sistema de estacionamiento medido, inspirada en el funcionamiento general del **SEOM de la ciudad de Santa Fe**.

En esta primera etapa **no vamos a utilizar HTML, CSS ni JavaScript**.

Primero vamos a construir la **lógica del sistema**. Más adelante vamos a tomar este mismo proyecto y transformarlo en una aplicación web utilizando las tecnologías de **Frontend** que trabajamos durante el primer cuatrimestre.

P SITUACIÓN



Nuestro sistema deberá trabajar con los datos de un vehículo:

Nombre del usuario

Patente

Saldo disponible

Número de box

Estado del estacionamiento

Cantidad de períodos utilizados

Para simplificar el ejercicio, cada período de estacionamiento tendrá un costo de:

💰 \$500

1 DATOS INICIALES

Primero deberás crear las variables necesarias para representar la información del usuario y del estacionamiento.

El programa deberá comenzar con:

un nombre;

una patente;

un saldo disponible;

un número de box;

el estado del estacionamiento;

un contador de períodos.

💡 **Pista:** revisa en la [Cartilla_PHP_BÁSICA_2026_A.PDF](#) cómo se declaran y utilizan las variables.

2 VALIDACIÓN DEL BOX



El sistema solamente acepta boxes comprendidos entre:

1000 y 1999

Deberás comprobar si el número ingresado es válido.

Si es válido, informar:

Box válido

Si no está dentro del rango:

Box no válido

💡 **Pista:** necesitas una condición que permita comprobar **dos situaciones al mismo tiempo**.

3 INICIAR EL ESTACIONAMIENTO

El sistema deberá comprobar si se puede iniciar el estacionamiento.

Para poder iniciarlo deberán cumplirse estas condiciones:

el box debe ser válido;

el usuario debe tener al menos **\$500** disponibles;

no debe existir otro estacionamiento activo.

Si todo está correcto, el estacionamiento deberá quedar **activo**.

Si alguna condición no se cumple, deberá mostrarse un mensaje indicando el problema.

💡 **Pista:** pensá qué condiciones deben analizarse y si alguna de ellas necesita estar dentro de otra.

4 SIMULAR LOS PERÍODOS

Si el estacionamiento está activo, deberás simular el paso de los períodos.

Cada período cuesta:

\$500

En cada repetición deberás:

descontar \$500 del saldo;

aumentar en 1 la cantidad de períodos;

mostrar el número del período;

mostrar el saldo restante.

El proceso deberá continuar **mientras exista saldo suficiente para pagar otro período.**

💡 **Pista importante:** pregúntate:

¿Sé desde el principio cuántos períodos voy a poder realizar?

Si la respuesta es **NO**, revisa la estructura while de la

[Cartilla_PHP_BÁSICA_2026_B.PDF](#).

5 FINALIZAR EL ESTACIONAMIENTO

Una vez finalizada la simulación, el estacionamiento deberá quedar inactivo.

El programa deberá informar:

que el estacionamiento finalizó;

el box utilizado;

la cantidad de períodos realizados.

6 CONSULTAR EL SALDO

El programa deberá mostrar el saldo disponible después de las operaciones realizadas.

Por ejemplo:

Saldo disponible: \$1500

💡 Recordá que el saldo debe modificarse cada vez que se descuenta o se carga dinero.



7 CARGAR SALDO

Ahora simularemos una carga de dinero.

Deberás crear una variable que represente una carga.

La carga solamente será válida si es:

mayor que 0

Si la carga es válida:

deberá sumarse al saldo;

deberá mostrarse el monto cargado;

deberá mostrarse el nuevo saldo.

Si la carga es 0 o negativa:

La carga debe ser mayor que cero.

8 MENÚ CON SWITCH

El sistema deberá tener una variable llamada **opción** que represente una acción del sistema.

Utiliza las siguientes opciones:

1 → Iniciar estacionamiento

2 → Finalizar estacionamiento

3 → Consultar saldo

4 → Cargar saldo

Deberás utilizar switch para determinar qué mensaje corresponde mostrar según el número de opción seleccionado.

Si se selecciona otro número:

Opción no válida

💡 **Pista:** revisa la estructura switch de la

[Cartilla_PHP_BÁSICA_2026_B.PDF](#).

9 PROCESAR 10 VEHÍCULOS

Para finalizar, el sistema deberá realizar una pequeña simulación.

Deberás procesar exactamente:

 **10 vehículos**

El programa deberá mostrar:

Procesando vehículo número 1

Procesando vehículo número 2

Procesando vehículo número 3

...

Procesando vehículo número 10

 **Pista:** en este caso conocemos exactamente cuántas veces debe repetirse el proceso.

Revisa la estructura for.

ANTES DE ESCRIBIR CÓDIGO

Antes de comenzar a programar, pensá y anota:

¿Qué variables necesito?

¿Qué valor inicial tendrá cada una?

¿Qué condiciones debe controlar el sistema?

¿Dónde necesito un if?

¿Dónde podría necesitar un if dentro de otro if?

¿Dónde conviene utilizar while?

¿Dónde conviene utilizar for?

¿Dónde puedo utilizar switch?

¿Qué variable necesito incrementar?

¿Qué condición hace que el while termine?

△ CONDICIÓN DE TRABAJO

Esta actividad tiene un objetivo muy concreto:

APRENDER A PENSAR LA LÓGICA DE UN PROGRAMA.

Por eso, para resolverla deberán utilizar únicamente:

Cartilla A

Cartilla B

Apuntes de clase;

Ejemplos realizados durante las clases;

Material disponible en Google Classroom.

⊘ NO UTILIZAR IA NI BUSCAR SOLUCIONES EN INTERNET.

Tampoco buscar en Internet un programa similar y adaptarlo.

La actividad está pensada para que ustedes construyan la solución desde cero.

No importa si al principio el programa no funciona.

Es parte del aprendizaje:

Probar → encontrar errores → pensar → corregir → volver a probar.

★ DESAFÍO EXTRA — OPCIONAL

Si terminaste correctamente el ejercicio obligatorio, podés intentar mejorar tu programa.

¿Podes hacer que el menú se repita?

La idea sería que el sistema continúe mostrando las opciones hasta que el usuario decida salir.

💡 Para resolverlo tendrás que combinar dos estructuras que ya conocemos:

while + switch

Este desafío es **opcional** y no es necesario para completar la actividad principal.

SEOM LAB

El objetivo de este trabajo no es copiar código.

Es aprender a transformar un problema en una solución lógica.

Primero pensamos. Después programamos.

Prof. Javier Imvinkelried

Programación I — 2.º año

Desarrollo de Software — ICOP